LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 6 – INHERITANCE

BILQIS KEYRA AZAHRA

103132400043

PS1DS – 05 – REG – 01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Inheritance (pewarisan) merupakan salah satu pilar utama dalam Pemrograman Berorientasi Objek (OOP) yang memungkinkan sebuah kelas diturunkan dari kelas lainnya. Dalam konsep ini, kelas yang menurunkan disebut sebagai superclass (kelas induk), sedangkan kelas yang diturunkan disebut subclass (kelas anak). Melalui inheritance, subclass akan mewarisi seluruh atribut (state) dan method (behavior) yang dimiliki oleh superclass, sehingga menghindari duplikasi kode dan menciptakan hierarki kelas yang lebih spesifik. Untuk mengakses anggota dari superclass (seperti memanggil constructor atau method induk), Java menyediakan keyword khusus yaitu super.

Selain inheritance, terdapat konsep Polymorphism yang memungkinkan suatu objek mengambil berbagai bentuk. Dua bentuk implementasi polymorphism yang dibahas dalam praktikum ini adalah Overloading dan Overriding. Overloading Method terjadi ketika dalam satu kelas terdapat dua atau lebih method dengan nama yang sama, namun memiliki jumlah atau tipe parameter yang berbeda. Sementara itu, Overriding Method terjadi ketika sebuah subclass mendefinisikan ulang method yang sudah ada pada superclass-nya dengan nama dan parameter yang persis sama; sehingga ketika method tersebut dipanggil melalui objek subclass, method milik subclass-lah yang akan dieksekusi.

B. Guided

1. Guided 01

A.java

```
package Guided.guided01;

public class A {
    int x;
    int y;

  void tampilXY(){
        System.out.println("Nilai x = "+x+", y = "+y);
    }
}
```

B.java

```
package Guided.guided01;

public class B extends A {
    int z;
    int hasil;

    void jumlahXYZ(){
        hasil = (x+y+z);
```

```
        System.out.println("Jumlah (x+y+z) = " + hasil );
    }

}
```

Demo.java

```
package Guided.guided01;

public class Demo {
    public class Demo {
      public static void main(String[] args){

        A superclass = new A();
        B subclass = new B();

        superclass.x = 3;
        superclass.y = 4;

        System.out.println("Superclass: ");
        superclass.tampilXY();

        subclass.x = 1;
        subclass.y = 2;
        subclass.z = 5;

        System.out.println("Subclass: ");
        subclass.jumlahXYZ();

    }
}
}
```

Output :

```
Nilai x = 3, y = 4
Subclass:
Jumlah (x+y+z) = 8
PS C:\Users\ACER\Documents\MATKUL\SMT 5\PBO\PRAKTIKUM\praktikum-pbo-ps1ds-05-reg-01>
```

2. Guided02

Demo02.java

```
package Guided.guided02.Demo.Demo02;
```

```java
public class Demo02 {
     public static void main(String[] args){
        Manager adi = new Manager("Adi", "Sukabiru");
        adi.departemen = "SDM";
        adi.tampilInformasi();
    }
}
```

Employee.java

```java
package Guided.guided02.Demo.Demo02;

public class Employee {
    private String name;
    String departemen;

    public Employee(String name){
        this.name = name;
    }

    public void tampilNama(){
        System.out.println("nama: "+ name);
    }
}
```

Manager.java

```java
package Guided.guided02.Demo.Demo02;

    public class Manager extends Employee {
    public String alamat;

    public Manager(String nama, String alamat){
        super(nama);
        this.alamat = alamat;
    }

    public void tampilInformasi(){
        super.tampilNama();
        System.out.println("alamat: " + alamat);
        System.out.println("departemen " + super.departemen);
    }
}
```

Output :

```
nama: Adi
alamat: Sukabiru
departemen SDM
PS C:\Users\ACER\Documents\MATKUL\SMT 5\PBO\PRAKTIKUM\praktikum-pbo-ps1ds-05-reg-01>
```

3. Guided 03

Lagu.java

```
package Guided.guided03;

public class Lagu {
     String pencipta;
    String judul;

    void IsiParam(String param1){
        judul = param1;
        pencipta = "Tidak dikenal";
    }

    void IsiParam(String param1, String param2){
        judul = param1;
        pencipta = param2;
    }
    void tampilkan(){
        System.out.println("Judul: "+ judul + ", pencipta: "+
pencipta);
    }
}
```

Main.java

```
package Guided.guided03;
public class Main {
     public static void main(String[] args) {
        Lagu d,e;
        d = new Lagu();
        e = new Lagu();

        d.IsiParam("Lagu 1");
        e.IsiParam("Lagu 2", "aku");
```

```
        d.tampilkan();
        e.tampilkan();
    }
}
```

Output :

```
Judul: Lagu 1, pencipta: Tidak dikenal
Judul: Lagu 2, pencipta: aku
PS C:\Users\ACER\Documents\MATKUL\SMT 5\PBO\PRAKTIKUM\praktikum-pbo-ps1ds-05-reg-01>
```

4. Guided 04

A.java

```
package Guided.guided04;
public class A {
     public void tampilkan() {
        System.out.println("method milik class A dipanggil..");
    }

}
```

B.java

```
package Guided.guided04;
public class B extends A {
    public void tampilkan(){
        super.tampilkan();
        System.out.println("method milik class b dipanggil..");
    }
}
```

Main.java

```
package Guided.guided04;
public class Main {
    public static void main(String[] args) {
        B overidB = new B();
        overidB.tampilkan();
    }
}
```

Ouput:

```
method milik class A dipanggil..
method milik class b dipanggil..
PS C:\Users\ACER\Documents\MATKUL\SMT 5\PBO\PRAKTIKUM\praktikum-pbo-ps1ds-05-reg-01>
```

C. Unguided

1. Soal Unguided 1

   Soal Unguided – Modul 6

   Buatlah sebuah program Java untuk menghitung estimasi biaya komputasi dari tiga jenis model machine learning, yaitu model regresi, model klasifikasi, dan model klasifikasi berbasis deep learning. Program ini terdiri dari satu kelas induk dan tiga kelas turunan, di mana seluruh atribut wajib bersifat private, sedangkan penamaan kelas, atribut, dan method dibebaskan.

   a. Kelas induk berfungsi sebagai model umum yang menyimpan nama model dan jumlah data latih. Kelas ini memiliki constructor, method untuk menampilkan informasi, serta dua method hitung biaya dengan nama yang sama (overloading): tanpa parameter yang menghitung data latih dikali 2, dan dengan satu parameter epoch yang menghitung data latih dikali epoch.

   b. Kelas turunan pertama adalah model regresi yang mewarisi kelas induk dan menambahkan atribut jumlah fitur. Kelas ini meng-override method tampilkan informasi dan hitung biaya tanpa parameter dengan rumus biaya induk ditambah jumlah fitur dikali 10. Selain itu, terdapat satu method baru untuk hitung biaya dengan dua parameter (epoch dan batch) yang menghitung data latih dikali epoch lalu dibagi batch.

   c. Kelas turunan kedua adalah model klasifikasi yang mewarisi kelas induk dan menambahkan atribut jumlah kelas. Kelas ini meng-override method tampilkan informasi dan hitung biaya tanpa parameter dengan rumus biaya induk dikali jumlah kelas, namun tidak meng-override method hitung biaya yang memiliki parameter epoch.

   d. Kelas turunan ketiga adalah model klasifikasi deep learning yang mewarisi kelas turunan kedua (bukan langsung kelas induk) dan menambahkan atribut jumlah layer. Kelas ini meng-override method tampilkan informasi dan hitung biaya tanpa parameter dengan rumus biaya dari kelas di atasnya ditambah jumlah layer dikali 200.

   e. Terakhir, pada class Main, buatlah tiga objek yaitu Regresi Linier (500 data latih, 4 fitur), Decision Tree (800 data latih, 3 kelas), dan Neural Network (1000 data latih, 5 kelas, 3 layer). Simpan ketiga objek tersebut ke dalam sebuah array bertipe kelas induk, lalu lakukan perulangan untuk menampilkan informasi masingmasing model, menghitung biaya komputasinya, dan menjumlahkan seluruh biaya.

Setelah perulangan selesai, tampilkan total biaya keseluruhan, serta tampilkan juga biaya 10 epoch untuk model Neural Network, biaya 10 epoch untuk model Regresi Linier, dan biaya 10 epoch dengan ukuran batch 50 untuk model Regresi Linier.

Jawaban :

ModelML.java

```java
package Unguided;

public class ModelML {
    private String namaModel;
    private int jumlahDataLatih;

    public ModelML(String namaModel, int jumlahDataLatih) {
        this.namaModel = namaModel;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    public String getNamaModel() { return namaModel; }
    public int getJumlahDataLatih() { return jumlahDataLatih; }

    public void tampilkanInformasi() {
        System.out.println("Nama Model : " + namaModel);
        System.out.println("Data Latih : " + jumlahDataLatih);
    }

    // Overloading method hitung biaya
    public double hitungBiaya() {
        return this.jumlahDataLatih * 2.0;
    }

    public double hitungBiaya(int epoch) {
        return this.jumlahDataLatih * epoch;
    }
}
```

RegresiLinear.java

```java
package Unguided;

public class RegresiLinier extends ModelML {
    private int jumlahFitur;

    public RegresiLinier(String namaModel, int jumlahDataLatih, int
jumlahFitur) {
        super(namaModel, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
```

```
    }

    @Override
    public void tampilkanInformasi() {
        super.tampilkanInformasi();
        System.out.println("Jumlah Fitur: " + jumlahFitur);
    }

    @Override
    public double hitungBiaya() {
        return super.hitungBiaya() + (this.jumlahFitur * 10.0);
    }

    // Method baru sesuai soal (b)
    public double hitungBiaya(int epoch, int batch) {
        return (super.getJumlahDataLatih() * (double)epoch) / batch;
    }
}
```

DeepLearning.java

```
package Unguided;

// Mewarisi dari Klasifikasi
public class DeepLearning extends Klasifikasi {
    private int jumlahLayer;

    public DeepLearning(String namaModel, int jumlahDataLatih, int
jumlahKelas, int jumlahLayer) {
        super(namaModel, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    @Override
    public void tampilkanInformasi() {
        super.tampilkanInformasi();
        System.out.println("Jumlah Layer: " + jumlahLayer);
    }

    @Override
    public double hitungBiaya() {
        return super.hitungBiaya() + (this.jumlahLayer * 200.0);
    }
}
```

Klasifikasi.java

```java
package Unguided;

public class Klasifikasi extends ModelML {
    private int jumlahKelas;

    public Klasifikasi(String namaModel, int jumlahDataLatih, int
jumlahKelas) {
        super(namaModel, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    @Override
    public void tampilkanInformasi() {
        super.tampilkanInformasi();
        System.out.println("Jumlah Kelas: " + jumlahKelas);
    }

    @Override
    public double hitungBiaya() {
        return super.hitungBiaya() * this.jumlahKelas;
    }
}
```

Main.java

```java
package Unguided;

public class Main {
    public static void main(String[] args) {
        // Mengcreate objek
        RegresiLinier regresi = new RegresiLinier("Regresi Linier",
500, 4);
        Klasifikasi decisionTree = new Klasifikasi("Decision Tree",
800, 3);
        DeepLearning neuralNetwork = new DeepLearning("Neural
Network", 1000, 5, 3);

        // Menyimpan dalam array bertipe kelas induk
        ModelML[] daftarModel = { regresi, decisionTree,
neuralNetwork };
        double totalBiayaKeseluruhan = 0;

        // Perulangan untuk menampilkan info & hitung biaya
        System.out.println("=== INFORMASI & BIAYA MODEL ===");
        for (ModelML model : daftarModel) {
            model.tampilkanInformasi();
```

```
            double biaya = model.hitungBiaya(); // Memanggil hitung
biaya tanpa parameter
            System.out.println("Biaya Komputasi Dasar : " + biaya);
            System.out.println("---------------------------------");
            totalBiayaKeseluruhan += biaya;
        }

        System.out.println("TOTAL BIAYA KESELURUHAN: " +
totalBiayaKeseluruhan);
        System.out.println();

        //Tampilkan biaya dengan parameter spesifik
        System.out.println("=== BIAYA BERDASARKAN EPOCH & BATCH
===");
        System.out.println("Neural Network (10 epoch) : " +
neuralNetwork.hitungBiaya(10));
        System.out.println("Regresi Linier (10 epoch) : " +
regresi.hitungBiaya(10));
        System.out.println("Regresi Linier (10 epoch, batch 50) : " +
regresi.hitungBiaya(10, 50));
    }
}
```

Output :

```
\redhat.java\jdt_ws\jdt.ls-java-project\bin' 'Unguided.Main'
=== INFORMASI & BIAYA MODEL ===
Nama Model : Regresi Linier
Data Latih : 500
Jumlah Fitur: 4
Biaya Komputasi Dasar : 1040.0
---------------------------------
Nama Model : Decision Tree
Data Latih : 800
Jumlah Kelas: 3
Biaya Komputasi Dasar : 4800.0
---------------------------------
Nama Model : Neural Network
Data Latih : 1000
Jumlah Kelas: 5
Jumlah Layer: 3
Biaya Komputasi Dasar : 10600.0
---------------------------------
TOTAL BIAYA KESELURUHAN: 16440.0

=== BIAYA BERDASARKAN EPOCH & BATCH ===
Neural Network (10 epoch) : 10000.0
Regresi Linier (10 epoch) : 5000.0
Regresi Linier (10 epoch, batch 50) : 100.0
PS C:\Users\ACER\Documents\MATKUL\SMT 5\PBO\PRAKTIKUM\praktikum-pbo-ps1ds-05-reg-01>
```

Penjelasan :

Berdasarkan oupput program, implementasi konsep Inheritance, Overriding, dan Overloading sudah berjalan sesuai dengan perintah. Program berhasil menghitung biaya komputasi spesifik untuk masing masing model sesuai rumus yaitu regresi linear, decision tree dan neural network dengan total keseluruhan biaya mencapai 16440. Selain itu, pengujian metode berparameter juga menunjukkan hasil yang tepat, seperti perhitungan biaya Neural Network untuk 10 epoch serta perhitungan khusus regresi linear yang menggunakan 10 epoch dan batch 50 yang mengjasilkan nilai 100.0

D. Kesimpulan

Berdasarkan praktikum yang telah dilaksanakan, dapat disimpulkan bahwa konsep Inheritance (pewarisan) sangat krusial untuk menciptakan hierarki kelas yang efisien dan menghindari duplikasi kode. Melalui hubungan superclass dan subclass, sebuah kelas turunan dapat mewarisi seluruh atribut dan method dari kelas induknya, serta memanfaatkan keyword super untuk mengakses anggota kelas induk tersebut. Praktikum ini juga membuktikan keberhasilan penerapan konsep Polymorphism melalui Overloading dan Overriding. Overloading memungkinkan pembuatan beberapa method dengan nama yang sama dalam satu kelas asalkan parameternya berbeda, sehingga meningkatkan fleksibilitas fungsi. Di sisi lain, Overriding memungkinkan subclass untuk memodifikasi perilaku method spesifik yang diwarisi dari superclass, sehingga suatu objek dapat merespons pemanggilan method yang sama dengan cara yang berbeda sesuai kebutuhannya. Secara keseluruhan, penguasaan kedua konsep ini memungkinkan praktikan untuk merancang program Java yang lebih modular, terstruktur, dan mudah dikembangkan.